

Table of Contents

1.Introduction.....	2
2. Objectives.....	2
3.Practical C Program: Process Information.....	3
4.Compilation and Execution.....	4
5.Finding C Header Files.....	5
6.Finding Compiled C Libraries.....	5
7.Static Linking.....	6
8.Dynamic Linking.....	7
9. ELF Headers (readelf-h).....	7
10.Program Headers — readelf -l.....	8
11.Dynamic Section — readelf -d.....	9
12.Checking Dependencies Using ldd.....	9
13.conclusion.....	10

Table of Figures

Figure 1: c program (procinfo.c).....	4
Figure 2: compile and execution.....	5
Figure 3: C header files.....	6
Figure 4: libc.so and libc.a commands.....	6
Figure 5: Static compilation, file size, and execution.....	7
Figure 6:Dynamic compilation, file size, and execution.....	8
Figure 7: readelf -h procinfo_dynamic.....	8
Figure 8: readelf -l procinfo_dynamic.....	9
Figure 9: readelf -d procinfo_dynamic.....	10
Figure 10:ldd procinfo_dynamic.....	10

Lab 2.1: C Libraries, Linking, and ELF Executable Structure

1. Introduction

When a C program uses library functions such as `printf()`, `getpid()`, and `time()`, several processes take place before and during execution. The compiler converts C source code into machine instructions, the linker resolves references to external libraries such as `libc`, and the operating system loads the resulting executable into virtual memory.

2. Objectives

The main objectives of this lab are:

- To understand how C programs use standard libraries.
- To understand the purpose of C header files.
- To identify the physical locations of C library files in Linux.
- To understand static and dynamic linking.
- To compile a C program using both static and dynamic linking.
- To compare static and dynamically linked executables.
- To understand the ELF executable format.
- To inspect ELF headers and program headers using `readelf`.
- To identify shared-library dependencies using `ldd`.

3. Practical C Program: Process Information

```
GNU nano 8.7.1                                procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {

    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}
```

Figure 1: c program (procinfo.c)

4.Compilation and Execution

After compiling and executing the program , It shows the possessed information that include T PID, parent PID.

```
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ gcc procinfo.c -o procinfo
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ./procinfo
=== Process Information ===
My Process ID (PID): 6645
Parent Process ID: 4019
Current Time: Sun Oct  4 15:08:34 2026
Executable Path: /proc/self/exe
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 2: compile and execution

5. Finding C Header Files

The commands show that the header files exist in: `/usr/include/`

The header files provide declarations that allow the compiler to understand how the functions should be called.

```
sunil@Ubuntu:/$ ls /usr
bin  games  include  lib  lib64  libexec  local  sbin  share  src
sunil@Ubuntu:/$ ls /usr/include
X11      elf.h      glob.h      misc      openvpn    semaphore.h  termios.h
aio.h    endian.h   gnu-versions.h  mntent.h   paths.h     setjmp.h     tgmth.h
aliases.h  envz.h     gnumake.h      monetary.h  poll.h      sgTTY.h      thread_db.h
alloca.h   err.h      grp.h          mqueue.h   printf.h     shadow.h      threads.h
ar.h       errno.h    gshadow.h      mtd         proc_service.h  signal.h     time.h
argp.h     error.h    iconv.h        net         protocols    sound         ttyent.h
argz.h     execinfo.h ifaddrs.h      netash      pthread.h    spawn.h       uchar.h
arpa       fcntl.h    inttypes.h     netatalk    pty.h        stab.h        ucontext.h
asm-generic features-time64.h iproute2      netax25     pwd.h        stdbit.h      ulimit.h
assert.h   features.h langinfo.h     netdb.h     python3.14   stdc-predef.h  unistd.h
byteswap.h  fenv.h     lastlog.h      neteconet   rdma         stdint.h      utime.h
c++        finclude   libgen.h       netinet     re_comp.h    stdio.h       utmp.h
complex.h  fmtmsg.h   libintl.h      netipx      regex.h      stdio_ext.h   utmpx.h
cpio.h     fnmatch.h  limits.h       netiucv     regexp.h     stdlib.h      values.h
crash      fstab.h    link.h         netpacket   regulator     string.h      video
crypt.h    fts.h      linux          netrom      resolv.h     strings.h     wait.h
ctype.h    ftw.h      locale.h       netrose     rpc          sudo_plugin.h  wchar.h
cxl        fwctl     malloc.h       nfs         rpcsvc       syscall.h     wctype.h
dirent.h   gconv.h    math.h         nl_types.h  sched.h      sysexits.h    wordexp.h
dlfcn.h    gdb        mcheck.h      nss.h       scsi         syslog.h      x86_64-linux-gnu
drm        getopt.h   memory.h       obstack.h   search.h     tar.h         xen
sunil@Ubuntu:/$ ls /usr/include/stdio.h
/usr/include/stdio.h
sunil@Ubuntu:/$ ls /usr/include/unistd.h
/usr/include/unistd.h
sunil@Ubuntu:/$ grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN. */
extern int fprintf (FILE *__restrict __stream,
sunil@Ubuntu:/$ grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
sunil@Ubuntu:/$
```

Figure 3: C header files

6. Finding Compiled C Libraries

```
sunil@Ubuntu:/$ ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
sunil@Ubuntu:/$ ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
```

Figure 4: libc.so and libc.a commands

7.Static Linking

In static linking, the linker (ld) copies the required machine code from .a archive files directly into the final executable at compile time.

```
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ gcc -static procinfo.c -o procinfo_static
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ls -lh procinfo_static
-rwxrwxr-x 1 sunil sunil 961K Oct  4 15:25 procinfo_static
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ./procinfo_static
=== Process Information ===
My Process ID (PID): 6858
Parent Process ID: 4019
Current Time: Sun Oct  4 15:25:25 2026
Executable Path: /proc/self/exe
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 5: Static compilation, file size, and execution

8.Dynamic Linking

In dynamic linking, the executable contains only references to shared libraries. The actual code is loaded into memory at runtime by the dynamic linker (typically `/lib64/ld-linux-x86-64.so.2`).

```
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ gcc procinfo.c -o procinfo_dynamic
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ls -lh procinfo_dynamic
-rwxrwxr-x 1 sunil sunil 16K Oct  4 15:26 procinfo_dynamic
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ./procinfo_dynamic
=== Process Information ===
My Process ID (PID): 6875
Parent Process ID: 4019
Current Time: Sun Oct  4 15:26:45 2026
Executable Path: /proc/self/exe
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 6:Dynamic compilation, file size, and execution

9. ELF Headers (readelf-h)

Global metadata about the file—is it a valid executable, and for which CPU?

```
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1140
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14264 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              14
  Size of section headers:               64 (bytes)
  Number of section headers:              31
  Section header string table index:     30
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 7: readelf -h procinfo_dynamic

10. Program Headers — readelf -l

Instructions for the OS loader—which segments to map into memory and with what permissions.

```
sunil@ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz            MemSiz             Flags   Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x0000000000000310 0x0000000000000310 R       0x8
INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374
               0x000000000000001c 0x000000000000001c R       0x1
               [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x00000000000007b8 0x00000000000007b8 R       0x1000
LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
               0x0000000000000311 0x0000000000000311 R E     0x1000
LOAD           0x0000000000002000 0x0000000000002000 0x0000000000002000
               0x00000000000001c0 0x00000000000001c0 R       0x1000
LOAD           0x0000000000002d80 0x0000000000002d80 0x0000000000002d80
               0x0000000000000290 0x0000000000000290 RW      0x1000
DYNAMIC        0x0000000000002d90 0x0000000000002d90 0x0000000000002d90
               0x00000000000001f0 0x00000000000001f0 RW      0x8
NOTE           0x0000000000000350 0x0000000000000350 0x0000000000000350
               0x0000000000000024 0x0000000000000024 R       0x4
NOTE           0x0000000000002170 0x0000000000002170 0x0000000000002170
               0x0000000000000030 0x0000000000000030 R       0x8
NOTE           0x00000000000021a0 0x00000000000021a0 0x00000000000021a0
               0x0000000000000020 0x0000000000000020 R       0x4
GNU_PROPERTY   0x0000000000002170 0x0000000000002170 0x0000000000002170
               0x0000000000000030 0x0000000000000030 R       0x8
GNU_EH_FRAME   0x0000000000002088 0x0000000000002088 0x0000000000002088
               0x0000000000000034 0x0000000000000034 R       0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000 RW      0x10
GNU_RELRO      0x0000000000002d80 0x0000000000002d80 0x0000000000002d80
               0x0000000000000280 0x0000000000000280 R       0x1

Section to Segment mapping:
Segment Sections...
00
01 .interp
02 .note.gnu.build-id .interp .gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela
t
03 .init .plt .plt.got .plt.sec .text .fini
04 .rodata .eh_frame_hdr .eh_frame .note.gnu.property .note.ABI-tag
05 .init_array .fini_array .dynamic .got .data .bss
06 .dynamic
07 .note.gnu.build-id
08 .note.gnu.property
09 .note.ABI-tag
10 .note.gnu.property
11 .eh_frame_hdr
12
13 .init_array .fini_array .dynamic .got
sunil@ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 8: readelf -l procinfo_dynamic

11.Dynamic Section — readelf -d

Metadata for the dynamic linker—which shared libraries must be loaded.

```
readelf: Error: Command not found
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
   Tag               Type              Name/Value
0x0000000000000001 (NEEDED)          Shared library: [libc.so.6]
0x000000000000000c (INIT)            0x1000
0x000000000000000d (FINI)            0x1304
0x0000000000000019 (INIT_ARRAY)      0x3d80
0x000000000000001b (INIT_ARRAYSZ)    8 (bytes)
0x000000000000001a (FINI_ARRAY)      0x3d88
0x000000000000001c (FINI_ARRAYSZ)    8 (bytes)
0x000000006ffffef5 (GNU_HASH)         0x390
0x0000000000000005 (STRTAB)          0x508
0x0000000000000006 (SYMTAB)          0x3b8
0x000000000000000a (STRSZ)           208 (bytes)
0x000000000000000b (SYMENT)          24 (bytes)
0x0000000000000015 (DEBUG)            0x0
0x0000000000000003 (PLTGOT)          0x3f80
0x0000000000000002 (PLTRELSZ)        192 (bytes)
0x0000000000000014 (PLTREL)          RELA
0x0000000000000017 (JMPREL)          0x6f8
0x0000000000000007 (RELA)           0x638
0x0000000000000008 (RELASZ)          192 (bytes)
0x0000000000000009 (RELAENT)         24 (bytes)
0x000000000000001e (FLAGS)           BIND_NOW
0x000000006ffffffb (FLAGS_1)         Flags: NOW PIE
0x000000006ffffffe (VERNEED)         0x5f8
0x000000006fffffff (VERNEEDNUM)      1
0x000000006ffffff0 (VERSYM)          0x5d8
0x000000006ffffff9 (RELACOUNT)       3
0x0000000000000000 (NULL)            0x0
```

Figure 9: readelf -d procinfo_dynamic

12.Checking Dependencies Using ldd

This shows the shared libraries required by the dynamically linked program and their locations.

```
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007fa6ddb41000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007fa6dd800000)
/lib64/ld-linux-x86-64.so.2 (0x00007fa6ddb43000)
sunil@Ubuntu:~/operating-system-security-and-networks/week3/lab2.1$
```

Figure 10:ldd procinfo_dynamic

13.conclusion

This lab provided a practical understanding of C libraries, linking, and ELF executable structure in Linux.

The experiment showed that C header files provide declarations required during compilation, while library files contain the compiled implementation of library functions. Static linking places required library code directly inside the executable, resulting in a larger and more self-contained program. Dynamic linking keeps library code separate and loads it at runtime, producing smaller executables with shared-library dependencies.

The `readelf` command was used to examine the ELF header, program headers, and dynamic section. The `ldd` command was used to identify the shared libraries required by a dynamically linked executable.

Overall, the lab demonstrated the complete relationship between C source code → compiler → linker → libraries → ELF executable → operating-system loader.

14. Knowledge Check Questions

Q1. What is the fundamental difference between a C header file and a shared library?

=>> A C header file contains declarations, function prototypes, macros, and type definitions.

⇒ A shared library contains compiled machine code that can be loaded at runtime by programs.

Q2. What is static and dynamic linking?

=> Static linking copies required library code into the executable during linking.

```
gcc -static procinfo.c -o procinfo_static
```

=> Dynamic linking keeps references to shared libraries in the executable. The libraries are loaded at runtime.

```
gcc procinfo.c -o procinfo_dynamic
```

Q3. What is an ELF Header? Write down the commands to check headers.

=> ELF stands for Executable and Linkable Format.

The ELF header contains important information about an executable, including:

- ELF identification/magic number
- 32-bit or 64-bit class
- Data encoding
- File type
- CPU architecture
- Entry point

The command used to view the ELF header is:

```
readelf -h procinfo_dynamic
```